Mixture of Experts and Agentic Post-Training at Scale

From Sparse Routing to Multi-Environment RL

CS 115B – Spring 2026

James Pustejovsky

April 14, 2026

Roadmap

Part I – Mixture of Experts: The Big Idea

- Why sparse models? The cost-vs.-capacity dilemma
- How routing works (with math, explained)
- Load balancing and why it matters
- From Mixtral to LatentMoE: a brief tour

Part II – Case Study: Nemotron 3 Super

- Architecture: Mamba + Attention + LatentMoE
- What makes it fast (speculative decoding, FP4 training)

Part III – Teaching LLMs to Act: Agentic Post-Training

- From last week's RLHF to multi-environment RL
- How agentic training data is made
- PivotRL and SWE-RL: two new ideas
- Open questions

The Dilemma

From our scaling laws lecture, we know:

$$\textit{More parameters} \rightarrow \textit{lower loss} \rightarrow \textit{better model.}$$

But more parameters also means:

- **More computation** per token (every parameter participates in every forward pass)
- **More memory** (all weights must be stored on GPUs)
- **Higher cost** at inference – and inference cost is paid *every query*

Can we have a model that knows a lot but doesn't think about everything at once?

This is the idea behind **Mixture of Experts**: a model with many parameters, but only a fraction are used for any given input.

An Analogy: The Hospital

Think of a hospital with many specialists:

- The hospital *employs* 500 doctors (total capacity)
- But for any one patient, a *triage nurse* (the **router**) selects 2–3 relevant specialists
- The patient doesn't see all 500 doctors – that would be slow and expensive
- The hospital's total knowledge is vast; the cost per patient is low

In a Mixture of Experts model:

- **Doctors** = expert networks (small FFNs)
- **Patient** = an input token
- **Triage nurse** = a learned routing function
- **Total capacity** = all experts combined ($\gg$ what any one token uses)

Where MoE Lives in the Transformer

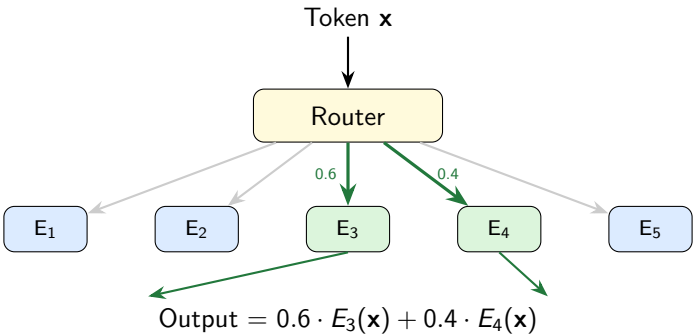
In a standard Transformer layer, there are two sublayers:



MoE replaces *only the FFN*. Attention is shared – every token still attends to every other token. The sparsity is purely in the "thinking" step, not the "looking around" step.

Each expert is itself a small FFN: $E_i(\mathbf{x}) = W_2^{(i)} \cdot \sigma(W_1^{(i)} \cdot \mathbf{x})$.

The Router: How Tokens Pick Experts



The router is a simple learned linear layer. It looks at the token and assigns a score to each expert. Only the top-k experts are activated; the rest are ignored.

The Math (Explained)

Step 1: Score every expert. The router is just a matrix multiply + softmax:

$$g(\mathbf{x}) = \text{softmax}(\mathbf{W}_g \cdot \mathbf{x}) \in \mathbb{R}^N$$

This gives a probability distribution over N experts. Think of it as: "How relevant is each expert to this particular token?"

Step 2: Pick the top k. Select the k experts with the highest scores. Zero out the rest.

Step 3: Combine. Run the token through each selected expert and mix the results:

$$\mathbf{y} = \sum_{i \in \text{top-}k} \hat{g}_i(\mathbf{x}) \cdot E_i(\mathbf{x})$$

where $\hat{g}_i$ are the scores renormalized to sum to 1 over the selected experts.

That's it. The router learns *which* experts to consult; the experts learn *what* to compute.

A Worked Example

Imagine $N = 8$ experts, $k = 2$.

Token $\mathbf{x}$ = the word "`derivative`" in a math context.

Router scores:

E_1	E_2	E_3	E_4	E_5	E_6	E_7	E_8
0.02	0.05	**0.41**	0.03	0.08	**0.31**	0.06	0.04

Top-2: E_3 (score 0.41) and E_6 (score 0.31).
Renormalize: $\hat{g}_3 = 0.41/(0.41 + 0.31) = 0.57, \quad \hat{g}_6 = 0.43.$
Output: $\mathbf{y} = 0.57 \cdot E_3(\mathbf{x}) + 0.43 \cdot E_6(\mathbf{x})$.

Experts $E_1, E_2, E_4, E_5, E_7, E_8$ are *never computed* for this token. We save 75% of the FFN computation.

The Numbers Game

Why does this matter at scale?

Model	Total Params	Active / Token	Ratio
Llama 3 70B (dense)	70B	70B	1×
Mixtral 8×7B	46.7B	12.9B	3.6×
DeepSeek-V3	671B	37B	18×
Nemotron 3 Super	120B	12B	10×

Nemotron 3 Super has the *knowledge capacity* of a 120B model but the *inference cost* of a 12B model.

The catch: You still need to *store* all 120B parameters in GPU memory. MoE saves compute, not storage.

The Load Balancing Problem

What goes wrong without intervention?

The router learns to send most tokens to the same few "popular" experts. This is called **expert collapse**.

Why it happens: Early in training, some experts get slightly better by chance. The router sends more tokens to them. They get even better (more training signal). Positive feedback loop.

Why it's bad:

- Most experts sit idle – wasted parameters
- The overloaded experts become a bottleneck on GPU utilization
- Effective model capacity shrinks to just a few experts

Analogy: Imagine if the triage nurse always sent every patient to the same two doctors. The hospital has 500 doctors but functions like a clinic with 2.

Fixing Load Balance

Solution 1: Auxiliary loss (GShard, Switch Transformer):
Add a penalty term to the training loss that discourages unbalanced routing:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{LM}} + \alpha \cdot \underbrace{N \sum_{i=1}^{N} f_i \cdot p_i}_{\text{load-balancing penalty}}$$

- f_i = fraction of tokens actually sent to expert i
- p_i = average router probability for expert i
- The product $f_i \cdot p_i$ is high when an expert is both popular and frequently chosen

Solution 2: Auxiliary-loss-free (DeepSeek-V3, Nemotron 3 Super):
Instead of a loss term, dynamically adjust expert bias terms during training. If an expert is underused, increase its bias; if overused, decrease it. No extra hyperparameter α to tune.

Modern trend is toward Solution 2 – more stable at very large scale.

Shared Experts: A Safety Net

Problem: Some information is useful for *every* token (basic syntax, common words). Forcing this through routed experts wastes routing decisions.

Solution (DeepSeek-V2, Nemotron 3 Super): Add "shared" experts that run on *every* token, alongside the routed ones:

$$\mathbf{y} = \underbrace{E^{\text{shared}}(\mathbf{x})}_{\text{always active}} + \underbrace{\sum_{i \in \text{top-}k} \hat{g}_i(\mathbf{x}) \cdot E_i(\mathbf{x})}_{\text{conditionally active}}$$

Think of it as: the shared expert handles the "common knowledge" that applies everywhere, freeing the routed experts to specialize in particular domains or skills.

A Brief Tour: Key MoE Models

Model	Lab	Total	Experts	Top-k	Innovation
Switch (2021)	Google	1.7T	2048	1	First trillion-param LM
Mixtral (2023)	Mistral	47B	8	2	Open-source MoE
DeepSeek-V3 (2024)	DeepSeek	671B	256	8	Aux-loss-free routing
Qwen3-235B (2025)	Alibaba	235B	128	8	Multi-env RL
Nemotron Super (2026)	NVIDIA	120B	512	22	LatentMoE + Mamba

Trends over five years:

- More experts, higher top-k (from top-1 to top-22)
- From auxiliary loss to auxiliary-loss-free balancing
- Post-training becoming as important as pre-training

Nemotron 3 Super: Three Ideas in One Model

NVIDIA's Nemotron 3 Super combines three architectural innovations:

1. LatentMoE
512 experts, top-22
via latent projection

2. Hybrid Mamba
Efficient sequence
modeling (no KV cache)

3. Multi-Token Prediction
Built-in speculative
decoding

Result: 120B total / 12B active parameters. Matches GPT-OSS-120B accuracy with 2.2× higher throughput. Supports 1M token context.

LatentMoE: The Key Insight

Standard MoE: Route tokens in the full hidden dimension d.
Problem: Routing traffic and expert weight reads both scale with d. This is the inference bottleneck.

LatentMoE: Project tokens to a *smaller* dimension ℓ before routing:

$$\mathbf{x} \in \mathbb{R}^d \xrightarrow{W_\downarrow} \mathbf{z} \in \mathbb{R}^\ell \xrightarrow{\text{route + experts}} \mathbf{z}' \xrightarrow{W_\uparrow} \mathbf{y} \in \mathbb{R}^d$$

Nemotron: $d = 4096$, $\ell = 1024$ ($4\times$ compression).

The trade: You saved $4\times$ on routing cost. Spend it on $4\times$ more experts:

- Standard MoE might use 128 experts, top-5
- LatentMoE uses 512 experts, top-22
- Same inference cost, but exponentially more expert *combinations*

More combinations = more specialized responses per token.

Why Mamba? The KV Cache Problem

Standard attention stores a key-value pair for every past token. For a 1M-token context, this becomes enormous.

Mamba (a state-space model) replaces most attention layers:

- Uses a *fixed-size state* instead of a growing KV cache
- Memory cost: $O(1)$ per layer during generation (vs. $O(T)$ for attention)
- Processes sequences in linear time (vs. $O(T^2)$ for attention)

But Mamba alone isn't enough. It can miss long-range dependencies that require token-to-token comparison.

Nemotron's solution: 88 layers, mostly Mamba, with a few attention "anchors" sprinkled in. The attention layers handle global routing; Mamba handles efficient local processing.

Multi-Token Prediction: A Built-In Draft Model

Standard LLM: Generate one token at a time (slow).

Speculative decoding: Use a small "draft" model to propose several tokens, then verify them all at once with the large model. Faster, but requires two separate models.

Nemotron's MTP: Train the model itself to predict the next M tokens at each position. The prediction heads act as the built-in draft model.

Innovation: Shared-weight heads. Instead of M independent heads (each predicting a fixed offset), use one head that is *reused* across positions. This makes it robust to its own mistakes when drafting autoregressively.

Result: On average, 3.45 tokens accepted per verification step (vs. 2.70 for DeepSeek-R1). Combined with NVFP4 quantization and sparse MoE, this yields the 2.2× throughput advantage.

Nemotron at a Glance

Property	Value
Total / Active params	120B / 12B
Experts / Active	512 / top-22 (LatentMoE)
Layers	88 (mostly Mamba + some Attention)
Training tokens	25 trillion
Max context	1M tokens
Training precision	NVFP4 (4-bit)
Speculative decoding	MTP shared-weight heads

Key comparisons (8K in / 64K out, B200 GPUs):

- vs. GPT-OSS-120B: comparable accuracy, 2.2× faster
- vs. Qwen3.5-122B: comparable accuracy, 7.5× faster

Speed comes from *all three innovations working together*: sparse MoE + Mamba (less memory) + MTP (fewer decoding steps) + FP4 (less bandwidth).

Recall: The Post-Training Pipeline

Last week we covered the standard pipeline:



This works well for making a helpful chatbot. But what if we want the model to *act in the world*?

- Write code and fix bugs in real repositories
- Use terminal commands to accomplish tasks
- Call APIs, search the web, process results
- Carry out multi-step plans over dozens of turns

These "agentic" capabilities require a fundamentally richer post-training recipe.

Nemotron's Post-Training Pipeline



Compare this to the standard 3-stage pipeline. Nemotron has *four* distinct RL stages after SFT:

1. **RLVR** – Multi-environment RL with verifiable rewards (math, code, tools, safety)
2. **SWE-RL** – End-to-end RL on real GitHub issues (the hardest environment)
3. **RLHF** – General instruction following and interaction quality

Each stage targets a different aspect of capability. The order matters: broad capability first, then specialized, then polish.

SFT: Two Losses, One Insight

Problem: When you train on a mix of long reasoning chains and short answers, the long outputs dominate the loss. The model gets great at reasoning but forgets how to give a brief, direct answer.

Nemotron's solution – two-stage SFT:

Stage 1 (token-level average): Every output token contributes equally.

$$\mathcal{L}_1 = \frac{\text{total loss over all output tokens}}{\text{total number of output tokens}}$$

Long outputs get proportionally more gradient signal. Good for learning reasoning.

Stage 2 (sample-level average): Every *conversation* contributes equally.

$$\mathcal{L}_2 = \frac{1}{\#\text{ conversations}} \sum_c \frac{\text{loss for conversation } c}{\text{output tokens in } c}$$

A 3-token answer counts as much as a 3000-token chain-of-thought. Restores balance.

Stage 1 teaches reasoning. Stage 2 teaches restraint.

The Agentic Data Explosion

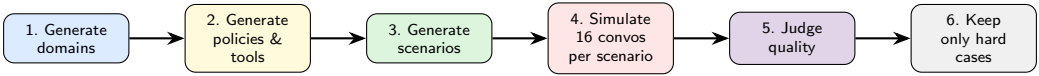
Nemotron 3 Super's SFT data is radically different from a standard chat dataset:

Category	Scale	What it teaches
Conversational Tool Use	279K convos	Multi-turn API calling
General Tool Use	1.5M traces	Diverse tool formats
Agentic Programming	28K tasks	CLI operation (Codex, etc.)
Software Engineering	–	Real GitHub issue resolution
Terminal Use	85K samples	Bash commands in Docker
Search Agent	–	Multi-hop web retrieval
CUDA Kernels	100K	GPU programming
Safety	–	Jailbreak resistance

Total: 7 million samples. The agentic portion is much larger than in any prior model.

How Agentic Data Is Made: A Six-Stage Pipeline

Example: Conversational Tool Use data generation



Step 4 is the key: three LLMs play different roles:

- **User LLM:** Simulates a customer with a specific scenario
- **Agent LLM:** Tries to help, calling tools as needed
- **Environment LLM:** Simulates tool execution and returns results

Step 6 is crucial: Drop scenarios where the agent always succeeds or always fails. Keep only the **informative** ones – where success requires skill.

From RLHF to RLVR

Last week we saw **RLHF**: RL from *Human* Feedback. A neural reward model scores outputs.

Nemotron primarily uses **RLVR**: RL from *Verifiable* Rewards.

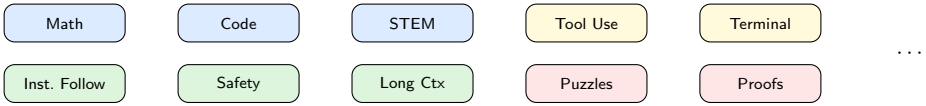
What's the difference?

	RLHF	**RLVR**
Reward signal	Neural network	Program / verifier
Reward noise	Noisy (model errors)	Exact (ground truth)
Reward hacking	Common	Rare
Domains	Open-ended chat	Tasks with clear answers
Scalability	Expensive (needs RM)	Cheap per evaluation

Examples: Math $\rightarrow$ check numerical answer. Code $\rightarrow$ run test cases. Tool use $\rightarrow$ check outcome. The reward is a binary "correct or not," computed by a program.

Multi-Environment RLVR

Nemotron trains on **21 environments simultaneously**:



Why all at once? When NVIDIA tried training on one environment at a time, the model improved on that task but **forgot** the others (catastrophic forgetting). Simultaneous training prevents this.

The algorithm is GRPO (from last week), but run **asynchronously**:

- Inference workers generate rollouts continuously
- Training engine updates the model when enough rollouts accumulate
- Updated weights are pushed to inference workers without waiting

Quick Refresher: GRPO

Recall from last week – Group Relative Policy Optimization:

For each prompt x: Generate G responses. Score them. Compute a relative advantage:

$$\hat{A}_i = \frac{r_i - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r})}$$

Intuition: "How much better (or worse) is this response compared to the group average?"

Update the model to increase the probability of above-average responses:

$$\mathcal{L} = -\frac{1}{G}\sum_i \min\Big(\rho_i \hat{A}_i,\ \text{clip}(\rho_i)\hat{A}_i\Big) - \beta \cdot \text{KL penalty}$$

Nemotron specifics: 256 prompts/step $\times$ 16 responses each. Max generation: 64K tokens. Because rollouts may use slightly stale model weights (async lag), they mask the importance ratio ρ_i to avoid instability.

Difficulty Curriculum

Not all prompts are equally useful for RL training.

Step 1 – Remove the easy stuff: If the SFT model already answers a prompt correctly every time, there's nothing to learn. Filter these out.

Step 2 – Sort by difficulty: Start training on easier problems, gradually introduce harder ones. This is standard curriculum learning applied to RL.

Step 3 – Special handling for "low-effort" mode: For some prompts, the reward considers both correctness *and* brevity:

$$r_{\text{low-effort}} = f(\text{correct}, \ |\text{tokens}|)$$

This teaches the model to give short, efficient answers when appropriate – not every question needs a 5000-token chain of thought.

PivotRL: Efficient Agentic RL

The problem: End-to-end RL for agentic tasks (tool use, code editing) is very expensive. Each rollout requires launching containers, executing code, waiting for results.

Pure SFT is cheap but brittle: Imitation learning from teacher trajectories works, but degrades on out-of-distribution inputs.

PivotRL (Yi et al., 2026) is a middle ground:

1. Start with offline expert trajectories (from teacher models)
2. Find "pivot turns" – turns where the model is *uncertain* about what to do next
3. Apply RL *only at those turns*, using a reward that credits functionally similar actions

Think of it as: "Don't re-learn what you already know. Focus your RL budget on the decisions that actually matter."

Result: Much cheaper than end-to-end RL, without the distribution-shift problems of pure SFT.

SWE-RL: The Hardest Environment

Task: Given a real GitHub issue, autonomously produce a code patch that passes all tests.

Each RL rollout:

1. Launch a container with the real codebase
2. The model interacts with a terminal: reads code, explores the repo, writes a patch
3. Run the ground-truth test suite on the patch
4. **Reward:** 1 if all tests pass, 0 otherwise

Why it's separate from RLVR: SWE rollouts take *minutes* (vs. seconds for math/code verification), use 10–50+ turns, and need up to 64K tokens. Mixing them with fast environments creates a throughput bottleneck.

Multi-harness training: The model trains with multiple CLI tool formats (OpenCode, Codex, etc.) – so it generalizes across agent frameworks at deployment.

RLHF: The Final Polish

After RLVR and SWE-RL, a final RLHF stage improves general interaction quality.

Innovation: Principle-following GenRM.

- Instead of a scalar reward model, use a generative model (Qwen3-235B) that *explains its judgment*
- The GenRM is conditioned on explicit behavioral principles (e.g., "be helpful but refuse unsafe requests")
- This allows fine-grained control over the model's behavior on safety and identity topics

Why not use RLHF for everything?

- RLHF rewards are noisy (neural network judges can be fooled)
- RLVR rewards are exact (programmatic verification)
- Use RLVR where you can verify; use RLHF where you can't

Safety Training: Attack and Defend

Two safety RL environments:

Anti-over-refusal: The model is rewarded for answering benign queries that *look* sensitive. Otherwise it learns to refuse anything remotely edgy.

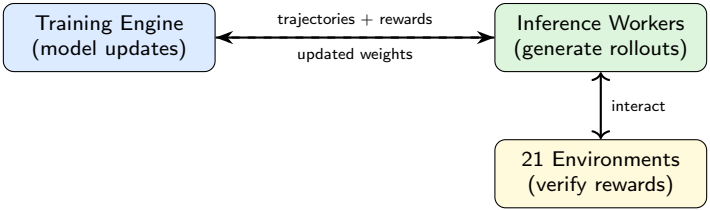
Anti-jailbreak: An adversarial loop:

1. Attack an early version of the model using automated prompt attacks (PAIR method)
2. Collect the prompts that successfully jailbreak the model
3. Add those prompts to the RL training data
4. The model learns to resist them
5. Repeat with stronger attacks

Additionally, Nemotron uses **deliberative alignment**: the model first generates a reasoning trace about *why* a request might be unsafe, then generates the response conditioned on that analysis. Think of it as "think before you speak" – but for safety.

The Infrastructure Challenge

Running async GRPO across 21 environments on 1000+ GPUs is an engineering challenge:



Key challenge: In-flight weight updates. The training engine pushes new weights to inference workers *without waiting* for ongoing rollouts to finish. A single trajectory can contain tokens from different model versions.

At 1000 GPUs, race conditions in port allocation became a major failure mode – solved with exclusive port claiming and coordinated initialization.

Results: Does It Work?

Benchmark	Nemotron Super (12B active)	GPT-OSS-120B (5B active)	Qwen3.5-122B (10B active)
SWE-Bench (coding)	**60.5**	59.9	41.9
HMMT Feb25 (math)	94.7	**95.4**	90.0
Terminal Bench Hard	22.8	**24.5**	19.0
Tau Bench v2 (tools)	**25.8**	–	24.0
IFBench (inst. follow)	72.6	**73.3**	68.3
HLE (science)	**18.3**	17.4	14.9
RULER @1M (long ctx)	**61.1**	–	61.0

Nemotron matches or exceeds much larger models on agentic benchmarks – **at** $2.2\times$ **the throughput**.

The SWE-Bench result (60.5%) reflects the dedicated SWE-RL stage. The Qwen3.5 gap on SWE-Bench (60.5 vs. 41.9) shows what multi-harness RL buys you.

The Recipe in Summary

1. **SFT**: 7M samples; two-stage loss (token-level → sample-level)

2. **RLVR**: 21 environments simultaneously; async GRPO; difficulty curriculum

3. **PivotRL**: Turn-level RL on offline expert traces for agentic tasks

4. **SWE-RL**: End-to-end RL in containerized repos; multi-harness training

5. **RLHF**: Principle-following GenRM for interaction quality and safety

Broad → specialized → polished. Each stage builds on the previous one.

This pipeline is considerably more complex than the standard SFT → RM → PPO recipe from just two years ago.

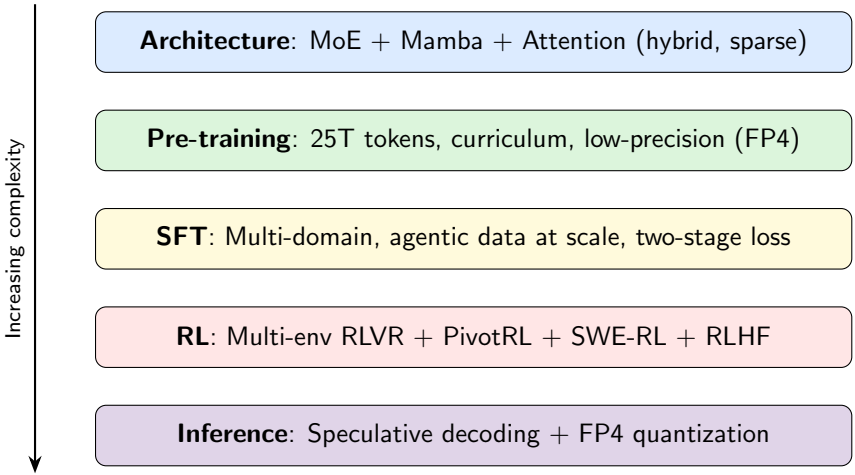
Comparing Approaches to Agentic Training

	SFT Only	**End-to-End RL**	**PivotRL**
Cost	Low	Very high	Medium
Needs live environments?	No	Yes	No
Out-of-distribution	Degrades	Robust	Robust
Long-horizon tasks	Moderate	Best	Good

PivotRL occupies the sweet spot for most agentic tasks. But for the hardest tasks (SWE-Bench), end-to-end RL is still needed.

Nemotron uses all three: SFT for initial capability, PivotRL for efficient agentic refinement, SWE-RL for the hardest problems.

The Emerging Stack



Observation: The post-training stack (SFT + RL) is now arguably *more complex* than pre-training. This is a significant shift from even two years ago.

Open Questions

1. **MoE scaling laws:** Do the Chinchilla rules apply to MoE? How should total experts, active experts, and data scale together?
2. **Environment scaling:** Is there a "scaling law" for the number of RL environments? When does adding more stop helping?
3. **Expert specialization:** Do individual experts develop interpretable specializations (e.g., a "math expert"), or is it more distributed?
4. **Agentic evaluation:** Current benchmarks test isolated tasks. How do we evaluate sustained, multi-session agent behavior?
5. **Data bottleneck:** Generating high-quality agentic training data is expensive and requires multi-model pipelines. Will this become the limiting factor?

Key Takeaways

1. **MoE** decouples model capacity from inference cost. The router selects which experts to use for each token; the rest are skipped.
2. **LatentMoE** projects tokens to a smaller dimension before routing, enabling many more experts at similar cost.
3. **Hybrid architectures** (Mamba + Attention) reduce memory overhead for long contexts while preserving global reasoning.
4. **Agentic post-training** requires multi-environment RL (not just chat RLHF), purpose-built data pipelines, and infrastructure at 1000+ GPUs.
5. **PivotRL** and **SWE-RL** represent two complementary strategies: efficient turn-level RL for most tasks, expensive end-to-end RL for the hardest.

For Next Time

Read: NVIDIA Nemotron 3 Super Technical Report, §3 (Post-Training).

References I

Primary reading:

NVIDIA. (2026). Nemotron 3 Super: Open, efficient Mixture-of-Experts hybrid Mamba-Transformer model for agentic reasoning. *Technical Report*.

MoE foundations:

Fedus, W., Zoph, B., & Shazeer, N. (2021). Switch Transformers: Scaling to trillion parameter models. *arXiv:2101.03961*.

Jiang, A.Q., et al. (2024). Mixtral of Experts. *arXiv:2401.04088*.

DeepSeek-AI. (2025). DeepSeek-V3 Technical Report. *arXiv:2412.19437*.

Elango, V., et al. (2026). LatentMoE. *NVIDIA Technical Report*.

Hybrid architectures and inference:

Dao, T., & Gu, A. (2024). Transformers are SSMs. *arXiv:2405.21060*.

Gloeckle, F., et al. (2024). Better & faster LLMs via multi-token prediction. *arXiv:2404.19737*.

RL and post-training:

Shao, Z., et al. (2024). DeepSeekMath: Pushing the limits of mathematical reasoning. *arXiv:2402.03300*.

Yi, J., et al. (2026). PivotRL: Turn-level reinforcement learning for agentic tasks. *Forthcoming*.

Guan, J., et al. (2025). Deliberative alignment: Reasoning enables safer language models. *arXiv:2412.16339*.